

IA no projeto — como e onde aplicar

Este documento explica, de forma prática, onde a IA entra no fluxo do app e como plugar isso no código que já existe. Não é uma proposta especulativa: usa o desenho que já está documentado em `organizacao/todo-list/src/todo_fintech_cloud.jsx` e o que a importação de extratos (`ImportarExtratoUseCase`) já deixou pronto para receber.

Onde a IA entra hoje

A importação de extrato (PDF/CSV/TXT/XLS/XLSX) já extrai os lançamentos e cria as transações — mas todas caem numa categoria genérica (`Outros` , tipo `AMBOS`) com `statusRevisao = PENDENTE_REVISAO` e `confiancaIa = null` . É exatamente aí que a IA entra: o próximo passo é **classificar essas transações pendentes na categoria certa**, preenchendo `confiancaIa` (0–100) para o usuário confiar ou não no resultado.

Upload do extrato

- `ImportarExtratoUseCase` (já existe)
- `Transacao(categoria="Outros", statusRevisao=PENDENTE_REVISAO)`
- `[AQUI] ClassificacaoService` ← o que falta construir
- `Transacao(categoria=X, statusRevisao=CLASSIFICADA, confiancaIa=87)`

A pipeline: cascata, não "manda tudo pra IA"

Chamar um modelo de linguagem para cada transação é caro e lento. A abordagem recomendada é uma cascata onde a IA só é acionada quando as etapas baratas falham:

1. **Regras do usuário** — se o usuário já classificou "UBER *TRIP" como "Transporte" antes, aplica direto (match exato, depois fuzzy/Levenshtein $\geq 85\%$). Confiança 100%, custo zero.
2. **Dicionário global** — tabela de ~200+ termos comuns (mercado, ifood, uber, netflix...) cacheada em Redis por 24h. Confiança alta, custo zero.
3. **Claude API** — só quando as duas etapas anteriores não resolveram. Envia a descrição normalizada + a lista de categorias disponíveis, recebe de volta a categoria sugerida e um grau de confiança.
4. **Fallback** `PENDENTE` — se a IA falhar ou não responder em 30s, a transação fica com `confiancaIa = 0` e categoria genérica, aguardando revisão manual.

Isso mantém o custo de IA baixo (só uma fração das transações chega até o passo 3) e o app continua funcionando mesmo se a API da IA cair.

Prompt: o que enviar (e o que nunca enviar)

- **Envie:** `descricaoNormalizada` da transação + a lista de categorias do usuário (nome + tipo).
- **Nunca envie:** CPF, nome completo, e-mail, número de conta ou qualquer dado que identifique a pessoa. O prompt deve carregar só o necessário para decidir a categoria — nada de LGPD sensível trafegando para uma API externa.
- **Resposta esperada:** categoria escolhida + confiança (0–100). Peça JSON estruturado (não texto livre) para evitar parsing frágil.

Onde plugar no código

Seguindo o padrão hexagonal já usado no projeto (mesma estrutura de

`ImportarExtratoUseCase`):

- **Porta:** `domain/classificacao/port/ClassificacaoIaPort` — interface simples, algo como `SugestaoCategoria sugerir(String descricaoNormalizada, List<Categoria> categorias)`. A porta não sabe se por trás tem Claude, OpenAI ou outra coisa.
- **Adapter:** `adapters/out/ia/ClaudeClassificacaoAdapter` — implementa a porta chamando a Claude API (Anthropic Messages API). Troca de provedor = troca de adapter, o use case não muda.
- **Use case:** `ClassificarTransacaoUseCase` — orquestra a cascata (regras → dicionário → `ClassificacaoIaPort` → fallback), igual ao que `ImportarExtratoUseCase` faz para extração de arquivo.
- **Auditoria:** registrar cada chamada (mesmo quando não usou IA) numa tabela `classificacao_logs`: estratégia usada, versão do modelo, versão do prompt, hash do prompt, resposta bruta, tokens usados, latência. Isso é o que permite medir taxa de acerto e custo depois.

Quando disparar a classificação

Duas opções, não excludentes:

- **Sob demanda:** botão "Classificar automaticamente" na tela de revisão do extrato (`/extratos/:id/revisar`), chamando o use case para as transações `PENDENTE_REVISAO` daquele extrato.
- **Assíncrono:** quando o RabbitMQ + `ProcessamentoJob` (já modelados no domínio, ver `TipoJob.classificacao_ia`) forem ligados de fato, a importação enfileira um job por extrato e um worker processa em background. É o desenho que N8N também pode disparar via webhook — ver o diagrama de arquitetura.

Custo e desempenho — cuidados práticos

- **Cache por descrição normalizada:** "UBER *TRIP 8342" de usuários diferentes tende a repetir. Cachear a resposta da IA por descrição (não por transação) evita pagar duas vezes pela

mesma pergunta.

- **Timeout curto (30s) com fallback:** nunca deixar o usuário esperando a IA indefinidamente — cai para `PENDENTE` e ele revisa manualmente.
- **Modelo:** para classificação de texto curto isto não precisa do modelo mais caro da linha — um modelo menor/rápido (ex.: Claude Haiku) tende a bastar; reserve um modelo maior só se a taxa de acerto do menor ficar baixa em produção.
- **Orçamento:** acompanhar `fintech_ia_classificacoes_corrigidas_total` (quantas sugestões da IA o usuário corrigiu manualmente) é a métrica que importa — não "quantas chamadas fizemos". Se a taxa de correção for alta, o prompt ou o modelo precisam de ajuste antes de gastar mais.

Resumo — o que fazer primeiro

1. Criar `ClassificacaoIaPort` + `ClaudeClassificacaoAdapter` (chamada real à API).
2. Criar `ClassificarTransacaoUseCase` com a cascata (regra → dicionário → IA).
3. Expor endpoint/botão para rodar a classificação nas transações `PENDENTE_REVISAO` de um extrato recém-importado.
4. Só depois disso investir em fila assíncrona (RabbitMQ) e automação via N8N — ver o diagrama de arquitetura N8N entregue junto com este documento para como essa camada se encaixa